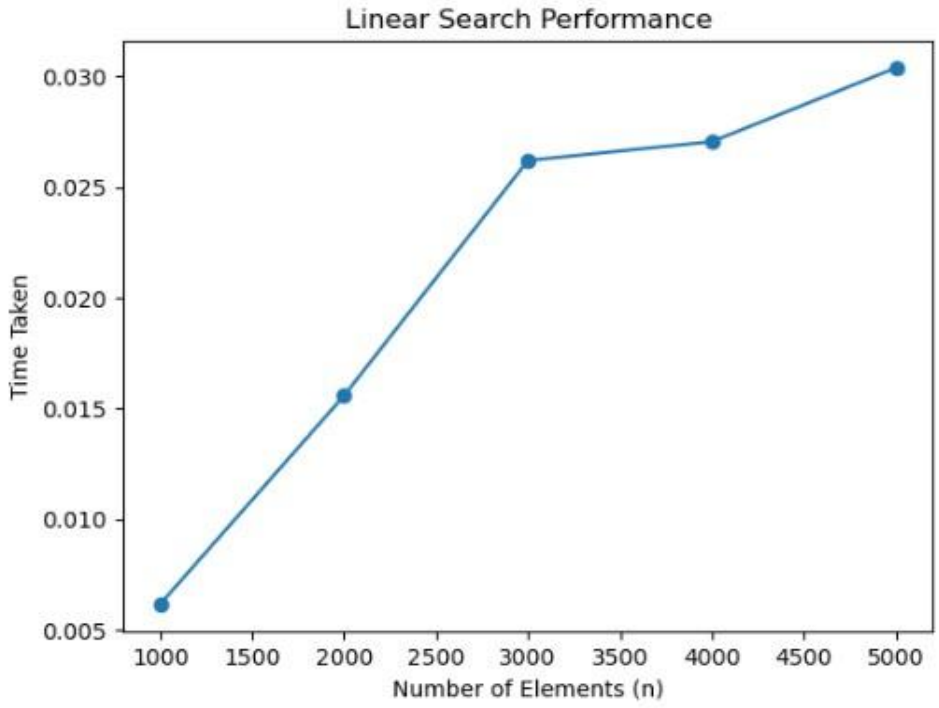
1. Write a program to implement linear search algorithm Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.

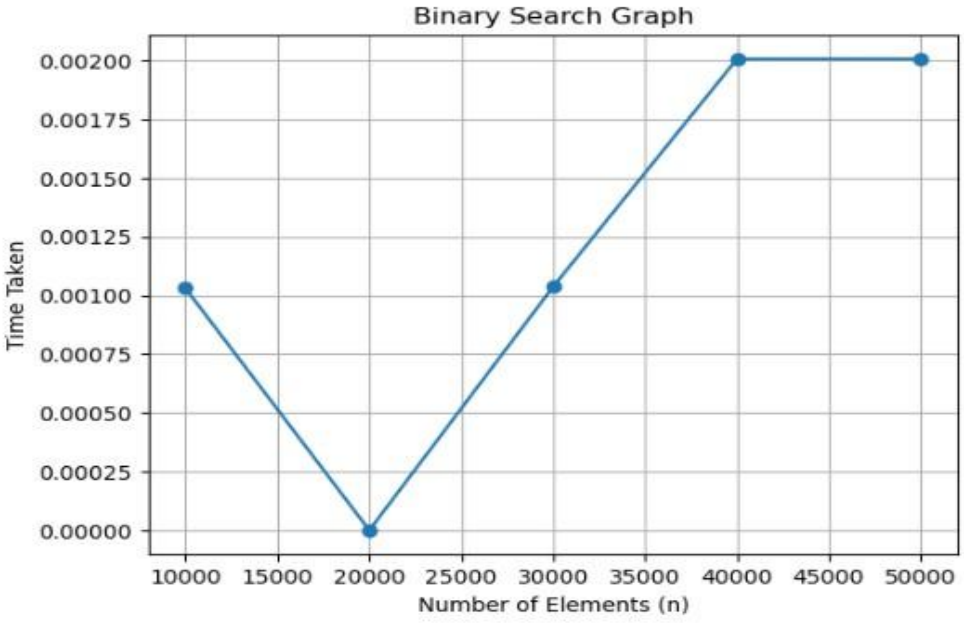
```
import time
import matplotlib.pyplot as plt

def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
    return -1

sizes = [1000, 2000, 3000, 4000, 5000]
times = []

for n in sizes:
    arr = list(range(n))
    key = n - 1

    start = time.time()
    for i in range(100):
        linear_search(arr, key)
    end = time.time()

    times.append(end - start)

plt.plot(sizes, times, marker='o')
plt.xlabel("Number of Elements (n)")
plt.ylabel("Time Taken")
plt.title("Linear Search Performance")
plt.show()
```



2. Write a program to implement binary search algorithm. Repeat the experiment for different values of n, the number of elements in the list to be searched and plot a graph of the time taken versus n.

```python
import time
import matplotlib.pyplot as plt

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1

sizes = [10000, 20000, 30000, 40000, 50000]
times = []

for n in sizes:
    arr = list(range(n))
    key = n - 1

    start = time.time()

    for i in range(100):
        binary_search(arr, key)

    end = time.time()

    times.append(end - start)


print("Time: ",times)
plt.plot(sizes, times, marker='o')
plt.xlabel("Number of Elements (n)")
plt.ylabel("Time Taken")
plt.title("Binary Search Graph")
plt.grid(True)
plt.show()
```

```
Time:  [0.00102996826171875, 0.0, 0.0010371208190917969, 0.0020067691802978516, 0.002007007598876953]
```



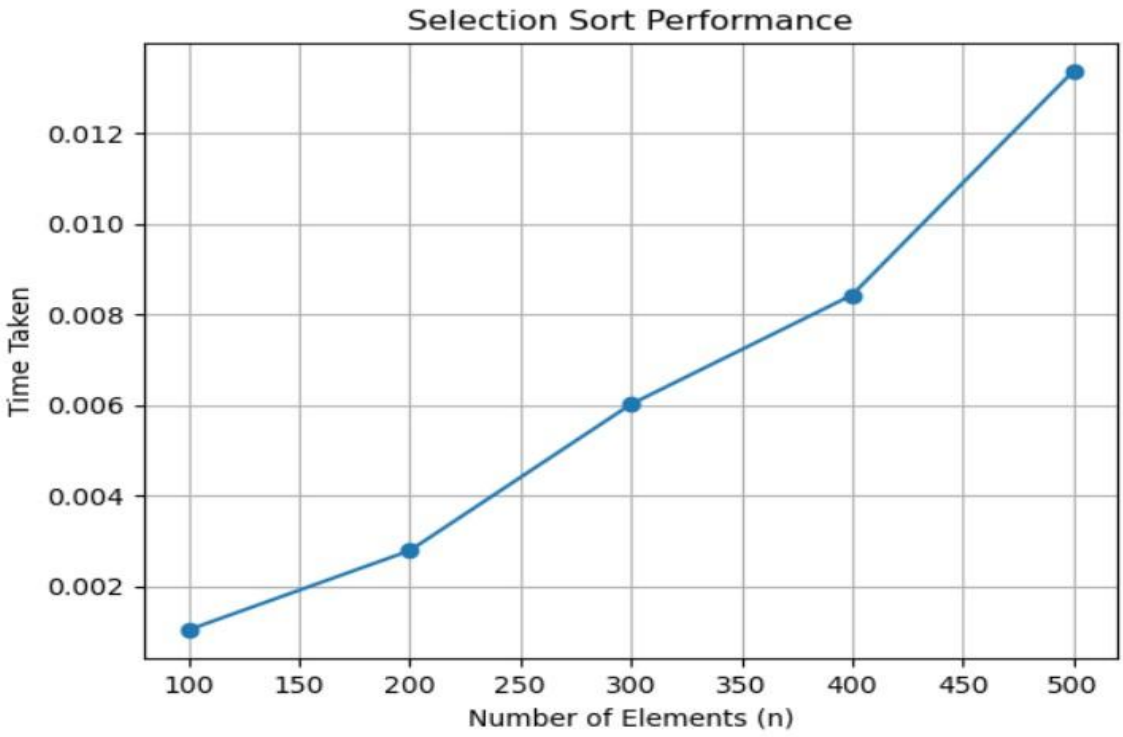
3. Write a program to solve towers of Hanoi problem and execute it for different number of disks.

```
def towers_of_hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return

    towers_of_hanoi(n - 1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    towers_of_hanoi(n - 1, auxiliary, target, source)

num_disks = int(input("Enter the number of disks: "))

towers_of_hanoi(num_disks, 'A', 'C', 'B')
```

```
Enter the number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

4. Write a Program to Sort a given set of numbers using selection sort algorithm. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n. The elements can be read from file or can be generated using the random number generator.

```
:
import random
import time
import matplotlib.pyplot as plt

def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

sizes = [100, 200, 300, 400, 500]
times = []

for n in sizes:
    arr = [random.randint(1, 1000) for i in range(n)]

    start = time.time()

    selection_sort(arr)

    end = time.time()

    times.append(end - start)

plt.plot(sizes, times)
plt.xlabel("Number of Elements (n)")
plt.ylabel("Time Taken")
plt.title("Selection Sort Performance")
plt.show()
```



m a

5.
Write a program to find minimum and maximum value in a given array using divide and conquer.

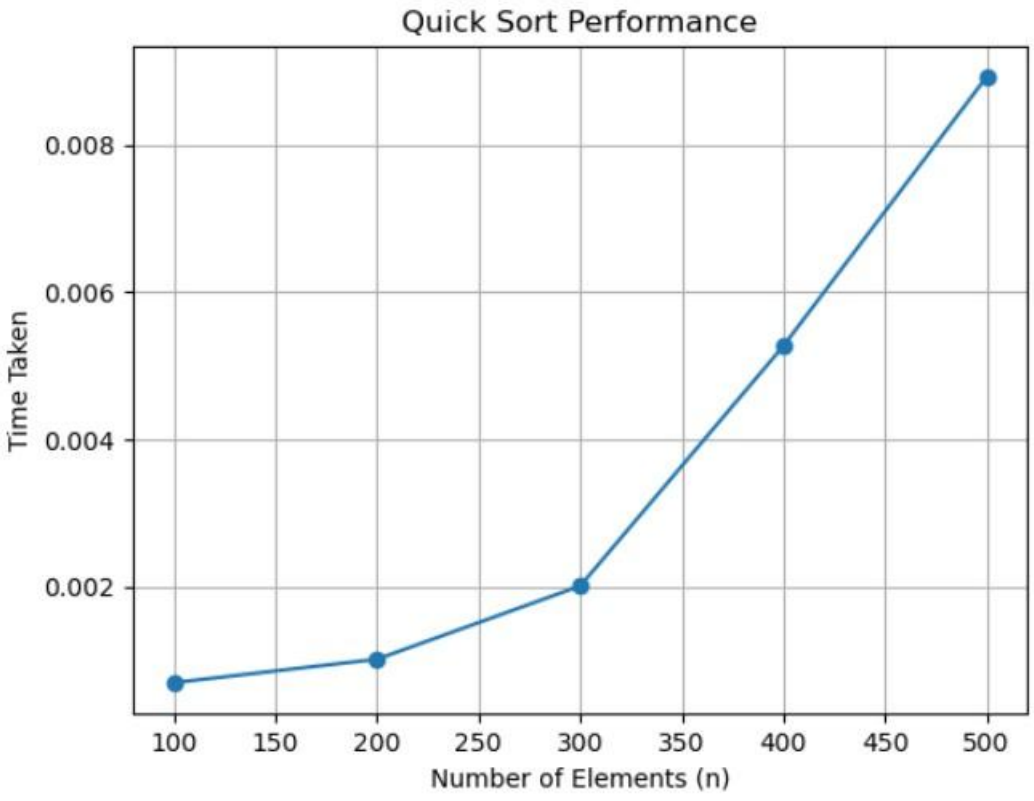
```python
# Find Minimum and Maximum using Divide and Conquer

def min_max(arr, low, high):

    if low == high:
        return arr[low], arr[low]

    mid = (low + high) // 2

    min1, max1 = min_max(arr, low, mid)
    min2, max2 = min_max(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)


arr = [12, 5, 8, 20, 3, 15]

minimum, maximum = min_max(arr, 0, len(arr) - 1)

print("Minimum value =", minimum)
print("Maximum value =", maximum)
```

```
Minimum value = 3
Maximum value = 20
```

6. Write a Program to Sort a given set of elements using quick sort algorithm. Repeat the experiment for different values of n, the number of elements in the list to be sorted and plot a graph of the time taken versus n.

6.

```python
import random
import time
import matplotlib.pyplot as plt

def quick_sort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[0]

    left = [x for x in arr[1:] if x <= pivot]
    right = [x for x in arr[1:] if x > pivot]

    return quick_sort(left) + [pivot] + quick_sort(right)

sizes = [100, 200, 300, 400, 500]
times = []

for n in sizes:
    arr = [random.randint(1, 1000) for i in range(n)]

    start = time.time()

    quick_sort(arr)

    end = time.time()

    times.append(end - start)

plt.plot(sizes, times, marker='o')
plt.xlabel("Number of Elements (n)")
plt.ylabel("Time Taken")
plt.title("Quick Sort Performance")
plt.grid(True)
plt.show()
```



7.

7. Write a program to implement dynamic programming algorithm for the optimal binary search tree.

```
freq = [34, 8, 50]
n = len(freq)

cost = [[0] * n for i in range(n)]

for i in range(n):
    cost[i][i] = freq[i]

for l in range(2, n + 1):
    for i in range(n - l + 1):
        j = i + l - 1
        cost[i][j] = 9999

        total = sum(freq[i:j + 1])

        for r in range(i, j + 1):
            left = cost[i][r - 1] if r > i else 0
            right = cost[r + 1][j] if r < j else 0

            cost[i][j] = min(cost[i][j], left + right + total)

print("Optimal Cost =", cost[0][n - 1])
```

```
Optimal Cost = 142
```

8. Write a program to implement Floyd's algorithm and find the length of the shortest paths from every pair of vertices in a given weighted graph.

```
INF = 999

graph = [
    [0, 5, 6, 10],
    [INF, 0, 3, INF],
    [6, INF, 0, 1],
    [INF, INF, INF, 0]
]

n = len(graph)

for k in range(n):
    for i in range(n):
        for j in range(n):
            graph[i][j] = min(graph[i][j], graph[i][k] + graph[k][j])

print("Shortest Distance Matrix:")
for row in graph:
    print(row)
```

```
Shortest Distance Matrix:
[0, 5, 6, 7]
[9, 0, 3, 4]
[6, 11, 0, 1]
[999, 999, 999, 0]
```

8.

9. Write a program to solve the string-matching problem using Boyer-Moore approach.

```
text = "ABCABCD"
pattern = "BCD"

m = len(pattern)
n = len(text)

shift = {}
for i in range(m - 1):
    shift[pattern[i]] = m - i - 1

i = m - 1

while i < n:
    j = m - 1
    k = i

    while j >= 0 and text[k] == pattern[j]:
        k -= 1
        j -= 1

    if j == -1:
        print("Pattern found at position", k + 1)
        break

    i += shift.get(text[i], m)
else:
    print("Pattern not found")
```

```
Pattern found at position 4
```

10. Write a program to implement BFS traversal algorithm.

```
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

queue = ['A']
visited = ['A']

while queue:
    node = queue.pop(0)
    print(node, end=" ")

    for i in graph[node]:
        if i not in visited:
            visited.append(i)
            queue.append(i)
```

```
A B C D E F
```

9.

11. Write a program to find minimum spanning tree of a given graph using Prim's algorithm.

```
import heapq

graph = {
    'A': [('B', 2), ('C', 3)],
    'B': [('A', 2), ('C', 1), ('D', 4)],
    'C': [('A', 3), ('B', 1), ('D', 5)],
    'D': [('B', 4), ('C', 5)]
}

visited = []
heap = [(0, 'A')]

while heap:
    w, node = heapq.heappop(heap)

    if node not in visited:
        visited.append(node)
        print((w, node))

        for n, wt in graph[node]:
            if n not in visited:
                heapq.heappush(heap, (wt, n))
```

```
(0, 'A')
(2, 'B')
(1, 'C')
(4, 'D')
```

12. Write a program to find minimum spanning tree of a given graph using Kruskal algorithm.

```
edges = [
    (1, 'B', 'C'),
    (2, 'A', 'B'),
    (3, 'A', 'C'),
    (4, 'B', 'D'),
    (5, 'C', 'D')
]

parent = {}

for _, u, v in edges:
    parent[u] = u
    parent[v] = v

def find(x):
    while parent[x] != x:
        x = parent[x]
    return x

for w, u, v in edges:
    if find(u) != find(v):
        print(u, "-", v, "=", w)
        parent[find(v)] = find(u)
```

```
B - C = 1
A - B = 2
B - D = 4
```

13. Write a program to find the subset of a given set = {s1, s2, sn} of n positive integers whose sum is equal to the given positive integer d. A suitable message is to be displayed if given problem instance doesn't have a solution. Check for if S = {1,2,5,6,8} and d = 9 and if S = {1,4,5,7} and d = 14

```
S = [1, 2, 5, 6, 8]
d = 9

dp = {0: [[]]}

for num in S:

    new_dp = dict(dp)

    for total in dp:

        new_total = total + num

        if new_total <= d:

            if new_total not in new_dp:
                new_dp[new_total] = []

            for subset in dp[total]:
                new_dp[new_total].append(subset + [num])

    dp = new_dp


if d in dp:

    print("Possible subsets are:")

    for subset in dp[d]:
        print(subset)

else:
    print("No subset found")
```

```
Possible subsets are:
[1, 2, 6]
[1, 8]
```